Software Test Plan

Supply Chain Management Simulator

Software Engineering Mini Project

Document ID	SCM-STP-001
Project	Supply Chain Management Simulator
Version	1.0
Authors	Siri Krishna C R, Spoorthi Poornachandra, D R Rushmitha, Kallepalli Sai Meghana, Shraddha
Date	11-10-2026
Status	Draft
Related documents	Software Requirements Specification v1.0 (SCM-SRS-001), Software Architecture and Design Document v1.0 (SCM-SAD-001)

Revision history

Version	Date	Author	Change summary
1.0	11-10-2026	Project team	Initial test plan: test items, features mapped to SRS requirements, approach, security validation, test cases and traceability

1. Introduction

Purpose: The purpose of this Software Test Plan is to define how the Supply Chain Management Simulator will be tested. Testing will verify inventory management, supplier selection, purchase orders, customer demand, order fulfilment, forecasting, supplier delivery delays, cost calculations, KPIs and reporting against the requirements in SRS v1.0.

Scope: Testing covers the command-line simulation of a single warehouse supplying multiple products to customers and replenishing stock from suppliers. It includes functional testing, boundary testing, invalid-input testing, integration testing, reproducibility testing, performance testing and verification of console and CSV reports.

The project uses simulated data. Real-world logistics, external supplier systems, multiple warehouses, graphical user interfaces, persistent databases and user authentication are outside the implementation scope described in the SRS and SAD, and are not tested.

References:

1. Software Requirements Specification (SRS), Supply Chain Management Simulator, Version 1.0 (SCM-SRS-001).
2. Software Architecture and Design Document (SAD), Supply Chain Management Simulator, Version 1.0 (SCM-SAD-001).
3. ISO/IEC/IEEE 29119-3:2021, Software and systems engineering: Software testing, Part 3: Test documentation. https://www.iso.org/standard/79429.html
4. Team GitHub repository scm-simulator and its continuous-integration workflow, .github/workflows/ci.yml.

Definitions:

Term	Meaning
Inventory	Goods currently held in stock in the warehouse
Stockout	When available stock is insufficient to fulfil demand
Backorder	Unfulfilled demand that is carried forward for later fulfilment
Lead time	Time between placing an order with a supplier and its expected delivery
Delivery delay	Extra days added to a purchase order when an unreliable supplier delivers late
FIFO	First In, First Out; earlier orders are fulfilled before later orders
EOQ	Economic Order Quantity; a calculated replenishment quantity intended to balance ordering and holding costs
ROP	Reorder Point; the stock level at which replenishment should be triggered
KPI	Key Performance Indicator; a measure used to assess the simulation's performance
MAE / MAPE	Measures used to evaluate forecasting errors
Seed	Starting value of the random number generator; the same seed gives the same run
Test case	A set of steps and an expected result used to check one particular behaviour
Defect	A problem or error in the system

Term	Meaning
CI	Continuous Integration; automated tests that run when code changes are submitted
RTM	Requirements Traceability Matrix

2. Test Items

The items under test are the Python modules of the simulator, at the commit on main that is tagged for each test cycle, together with the sample scenario and the CI workflow.

Test item	File	Covered by tests in
Product catalog	src/products.py	tests/test_products.py
Warehouse inventory and replenishment calculations	src/inventory.py	tests/test_inventory.py
Cost tracking	src/costs.py	tests/test_costs.py
Supplier management and selection	src/suppliers.py	tests/test_suppliers.py
Purchase orders and supplier delays	src/purchase_orders.py	tests/test_purchase_orders.py
Customer demand generation	src/demand.py	tests/test_demand.py
Customer order management and fulfilment	src/orders.py	tests/test_orders.py
Demand forecasting	src/forecasting.py	tests/test_forecasting.py
Simulation clock and event log	src/clock.py	tests/test_clock.py
Configuration loading and validation	src/config_loader.py	tests/test_config_loader.py
Inbound shipments and transport cost	src/shipments.py	tests/test_shipments.py
KPI calculation	src/kpis.py	tests/test_kpis.py
CSV export and summary report	src/reports.py	tests/test_reports.py
Simulation engine and command-line interface	src/simulation.py, src/main.py	tests/test_simulation.py, tests/test_nfr.py
Sample scenario	data/sample_config.json	tests/test_config_loader.py

3. Features to be Tested

Features are listed by their SRS requirement ID.

Req ID	Feature	What we will check
SCM-F-001	Product data	Valid products are created; empty IDs and negative costs are rejected
SCM-F-002	Product catalog	Duplicate IDs are rejected; products are listed in ID order
SCM-F-003	Inventory update	Stock rises on receipt, falls on shipment and never goes below zero
SCM-F-004	Economic order quantity	EOQ matches the formula, including zero demand and invalid holding cost
SCM-F-005	Reorder decision	Safety stock and reorder point values; reorder is triggered only at or below the reorder point
SCM-F-006	Cost tracking	Each cost category and the total are calculated correctly
SCM-F-010	Supplier data	Invalid lead time, reliability and prices are rejected
SCM-F-011	Supplier lookup	The right suppliers are returned for each product
SCM-F-012	Supplier selection	Cheapest, fastest and balanced strategies pick the expected supplier
SCM-F-013	Purchase order creation	Price, expected delivery day and quantity limits are correct
SCM-F-014	Purchase order tracking	Only allowed status changes happen; orders arrive on their due day

Req ID	Feature	What we will check
SCM-F-015	Delivery delay	Unreliable suppliers deliver late; delays are repeatable with the same seed
SCM-F-020	Demand generation	Constant, uniform and seasonal patterns give the expected values
SCM-F-021	Order creation	Customer orders record quantity, shipped quantity and status
SCM-F-022	Order book	Orders are numbered in sequence and listed oldest first
SCM-F-023	Stock validation and backorders	Orders exceeding available stock are partly filled and the rest is backordered, oldest order first
SCM-F-024	Moving average forecast	Forecast and demand variability match hand-calculated values
SCM-F-025	Exponential smoothing and forecast error	Smoothed forecast, MAE and MAPE match hand-calculated values
SCM-F-030	Simulation clock and events	The run stops after the configured days; events are recorded by type
SCM-F-031	Data validation	Missing sections, unknown products, out-of-range values and invalid JSON are rejected
SCM-F-032	Transport cost	Cost by mode and distance is correct; unknown modes are rejected
SCM-F-033	KPIs	Fill rate, service level, average inventory and turnover are correct
SCM-F-034	Reporting	CSV rows and the printed summary are accurate
SCM-F-035	End-to-end simulation	A full run produces consistent daily records and correct exit codes
SCM-NF-001	Performance	A 365-day run finishes in under 2 seconds
SCM-NF-002	Repeatability	The same configuration and seed give identical output
SCM-NF-003	Testability	Every functional requirement has an automated test; CI finishes in under 3 minutes
SCM-NF-004	Usability of errors	Input errors give one clear line and exit code 2
SCM-NF-005	Portability	Only standard library modules are imported; tests pass on Windows and Ubuntu
SCM-SR-001 to 005	Security requirements	See Section 5.1

4. Features Not to be Tested

- Actual physical transportation of goods.
- Real supplier or courier operations, and external warehouse or supplier systems; none are integrated.
- Real-time GPS tracking and shipment tracking after dispatch; the system does not implement them.
- Multiple warehouses, graphical user interfaces, databases and user login; these are out of scope in the SRS.
- Runs longer than 3650 days or configuration files larger than 1 MB, beyond checking that they are rejected.
- Features not included in the approved SRS.

5. Test Approach / Strategy

Levels:

Type	Explanation	Example in our project
Unit testing	Test one part separately	Check that shipping 8 units with only 5 in stock ships 5 and leaves stock at 0 (TC-INV-03)

Type	Explanation	Example in our project
Integration testing	Check whether parts work together	Run the engine with all modules for 30 days and check that no stock or backorder goes negative (TC-SIM-02)
System testing	Test the complete system through its user interface	Run the command line with a missing configuration file and check the error and exit code (TC-SIM-06)
Acceptance testing	Check whether the system meets the requirements	Walk through use cases UC-1.1 and UC-2.1 to 2.6 from the SRS at the demo, using the sample scenario

Types:

- Functional testing: check whether each feature works as expected, using the values in the SRS acceptance criteria.
- Boundary testing: check quantities at important limits, such as zero stock, an order equal to available stock, reliability 0.0 and 1.0, and 1 and 3650 days.
- Negative testing: try invalid inputs, such as a negative quantity, a missing product ID, an unknown transport mode or malformed JSON.
- Performance testing: time a 365-day run of the sample scenario.
- Regression testing: the full suite runs in GitHub Actions on every pull request and every push to main, so earlier tests are repeated after every change.
- Usability testing: check that the printed summary and error messages are clear and complete.

Automation: all test cases are automated with pytest. Each test function starts its docstring with its test case ID, so results can be traced to this plan. Line and branch coverage are measured with pytest-cov.

Entry criteria:

- The features to be tested in the cycle have been merged to main.
- The application can be started successfully with the sample scenario.
- Test data for products, suppliers, demand and policy is available in data/sample_config.json.
- The CI workflow runs on main without errors.
- The expected result for each test case is defined in this plan or in the SRS acceptance criteria.

Exit criteria:

- All planned test cases have been executed.
- All High-priority test cases pass and no critical defect remains open.
- Line coverage of src/ is at least 80%.
- Failed test cases have been recorded, fixed and re-run.
- The test results have been documented in the test reports.

5.1 Security Validation

Security validation checks each security requirement in SRS 5.1.2 directly.

Req ID	What is validated	Method	Test case / evidence
SCM-SR-001	Invalid configuration values and references are rejected before the run starts	Negative tests with a missing section, unknown product, product with no supplier and malformed JSON	TC-CFG-02 to 06
SCM-SR-002	Oversized input and excessive run length are rejected	Configuration file of 1.1 MB; days set to 0 and 5000	TC-CFG-06

Req ID	What is validated	Method	Test case / evidence
SCM-SR-003	CSV formula injection is neutralised	Export a record whose product ID starts with =; check the cell starts with an apostrophe and that numbers are unchanged	TC-RPT-03, TC-RPT-04
SCM-SR-004	Main branch is protected	Attempt a direct push to main; inspect the ruleset (PR required, 1 approval, required check, no bypass)	Rejected push message and ruleset screenshot
SCM-SR-005	Dependencies are pinned and no secrets are committed	Review requirements.txt for exact versions and .gitignore for .env and virtual environments; checked in every pull request review	Review checklist

6. Test Environment

Item	Details
Hardware	Team laptops used for development and testing; GitHub-hosted runner for CI
Software	Windows 10/11 on team laptops; Ubuntu (ubuntu-latest) on CI; Python 3.12
Database	None; the system does not use a database
Tools	VS Code for development, pytest 8.3.3 for automated testing, pytest-cov for coverage, GitHub for version control and pull requests, GitHub Actions for CI, GitHub Issues for defect reporting
Test data	data/sample_config.json (3 products, 3 suppliers, 3 demand patterns, 3 transport modes) and configuration variants created by the tests in temporary folders

7. Test Schedule

Milestone	Date
Test plan and test case design baseline	11-10-2026
CI environment ready on the team repository	Before Sprint 1
Sprint 1: unit tests written and run with each feature	07-11-2026 to 08-11-2026
Sprint 2: remaining unit tests, integration, system and NFR tests	09-11-2026 to 10-11-2026
Code freeze	10-11-2026, 11:59 PM
Final test execution and test reports	11-11-2026
Acceptance (demo)	After 11-11-2026, as scheduled by the course

8. Test Deliverables

- Software Test Plan (this document).
- Automated test cases in the tests/ folder.
- Test data: data/sample_config.json.
- Test execution results: CI run logs on GitHub Actions.
- Defect reports as GitHub Issues.
- Unit Test Report, System Validation Report, Code Coverage Report and Branch Coverage Report in docs/reports/.

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	D R Rushmitha	Prepare the Test Plan, coordinate testing activities, track test progress and prepare the test summary report.
Test Engineer	Spoorthi Poornachandra	Design and execute the integration and system test cases, record test results and report defects.
Test Engineer	Siri Krishna C R	Design and execute test cases, analyse coverage results and report defects.
Developer	All five members	Each member writes the unit tests for the features they implement, in the same pull request as the code, and fixes defects in their modules.
Test support	Shraddha	Prepares the demo scenario files used for acceptance testing and keeps the sprint test notes.
Product Owner	Kallepalli Sai Meghana	Confirm requirements, review and merge pull requests as repository owner, review results and approve readiness.

10. Risks and Mitigation

Risk	Mitigation
Incorrect test data	Use the shared sample scenario and hand-calculated expected values from the SRS acceptance criteria
Integration problems between modules	Test modules individually first; keep module interfaces fixed as in SAD 4.3
Insufficient time for testing	Write each feature's tests in the same pull request as the feature
Bugs discovered late	CI runs the full suite on every pull request, so failures appear the day the code is written
Results differ between runs	All randomness uses one seeded generator; a repeatability test compares two runs
Large runs slow the simulation	Time a 365-day run (TC-NFR-01) and cap runs at 3650 days

11. Assumptions & Dependencies

Assumptions:

- Sample supplier, product and demand data will be available in data/sample_config.json.
- Inventory quantities will have a defined starting value in the scenario.
- Delivery delays are simulated from supplier reliability with a fixed seed, so expected results can be stated.
- GitHub Actions will be available to run the test suite.

Dependencies:

- The inventory and order modules must be ready before the engine integration tests can run.
- Purchase order and supplier modules must be ready before reorder and delay behaviour can be tested end to end.
- The command-line interface must be ready before system tests can run.

12. Suspension & Resumption Criteria

Suspend testing if:

- The application cannot start or crashes repeatedly.
- The CI environment becomes unavailable.
- A major defect prevents most planned tests from being executed.
- Test data is corrupted or unreliable.

Resume testing if:

- The blocking defect has been fixed and merged.
- The application and CI are working again.
- Test data has been checked and restored if necessary.
- The affected tests can be executed reliably.

13. Test Case Management & Traceability

Test cases are stored as pytest functions in tests/. Each function's docstring starts with its test case ID. Defects are logged as GitHub Issues and linked to the pull request that fixes them. The full RTM is in SRS Section 8; Section 13.2 below maps every requirement to its test cases.

13.1 Test cases

The key test cases are specified below. The remaining test cases in each range follow the same format and are listed with their requirement in Section 13.2.

TC ID	Req ID	Level / Type	Precondition / Test data	Steps	Expected result
TC-INV-03	SCM-F-003	Unit / Boundary	Warehouse with 5 units of P-001	Ship 8 units of P-001	Returns 5; stock of P-001 is 0
TC-INV-07	SCM-F-004	Unit / Functional	None	Call economic_order_quantity(1000, 50, 2)	Returns 224
TC-INV-12	SCM-F-005	Unit / Functional	None	Call reorder_point(10, 4, 14)	Returns 54
TC-INV-14	SCM-F-005	Unit / Boundary	Stock 50 of P-001	Call needs_reorder("P-001", 54, on_order=10)	Returns False
TC-SUP-12	SCM-F-012	Unit / Functional	Suppliers S-001 and S-002 from the sample scenario	select_supplier for P-001 with strategy balanced	Returns S-002 (score 0.606 against 0.760)
TC-PO-01	SCM-F-013	Unit / Functional	Supplier S-001 (lead time 3)	Create a purchase order on day 5	Expected delivery day is 8
TC-PO-09	SCM-F-014	Unit / Boundary	Purchase order due on day 8	Call due_on(7), then due_on(8)	Not returned on day 7; returned on day 8
TC-PO-12	SCM-F-015	Unit / Boundary	random.Random(42); 100 open orders	apply_random_delay with reliability 0.0 and maximum delay 3	Every delay is between 1 and 3 days
TC-ORD-14	SCM-F-023	Unit / Functional	Open orders of 5 units (day 1) and 5 units (day 2)	allocate(product, 7)	Returns 7; first order FULFILLED; second PARTIAL with 2 shipped
TC-FC-06	SCM-F-025	Unit / Functional	None	exponential_smoothing([10, 20, 30], 0.5)	Returns 22.5
TC-CFG-04	SCM-F-031, SCM-SR-001	Unit / Negative	Copy of sample config with demand for P-999	load_config on the copy	Rejected with ValueError naming P-999
TC-CFG-06	SCM-SR-002	Unit / Negative	Copies of sample config with days 0 and 5000, and a 1.1 MB file	load_config on each	Each is rejected with ValueError
TC-RPT-03	SCM-F-034, SCM-SR-003	Unit / Security	One daily record with product ID "=HYPERLINK(1)"	export_daily_csv to a temporary file; read it back	Cell reads "'=HYPERLINK(1)"; numeric cells unchanged

TC ID	Req ID	Level / Type	Precondition / Test data	Steps	Expected result
TC-SIM-01	SCM-F-035	Integration	Sample scenario	Run the simulation for 30 days	90 daily records (30 days x 3 products)
TC-SIM-06	SCM-F-035, SCM-NF-004	System / Negative	No file named missing.json	main(["--config", "missing.json"])	Returns exit code 2; one line on stderr starting "Error:"
TC-NFR-01	SCM-NF-001	System / Performance	Sample scenario	Time a 365-day run with time.perf_counter	Completes in under 2.0 seconds
TC-NFR-02	SCM-NF-002	System / Repeatability	Sample scenario, seed 42	Run twice, writing two CSV files	The two files are byte-identical
TC-NFR-03	SCM-NF-005	Static / Portability	Source files in src/	Collect every imported top-level module	Each is src or in the Python standard library

13.2 Requirement to test case mapping

Req ID	Requirement	Test case IDs
SCM-F-001	Product data and validation	TC-PRD-01 to 04
SCM-F-002	Product catalog	TC-PRD-05 to 10
SCM-F-003	Warehouse stock	TC-INV-01 to 06
SCM-F-004	Economic order quantity	TC-INV-07 to 10
SCM-F-005	Safety stock, reorder point, reorder check	TC-INV-11 to 15
SCM-F-006	Cost tracking	TC-CST-01 to 04
SCM-F-010	Supplier data and validation	TC-SUP-01 to 05
SCM-F-011	Supplier registry	TC-SUP-06 to 09
SCM-F-012	Supplier selection	TC-SUP-10 to 15
SCM-F-013	Purchase order creation	TC-PO-01 to 05
SCM-F-014	Purchase order status and order book	TC-PO-06 to 10
SCM-F-015	Supplier delivery delays	TC-PO-11 to 14
SCM-F-020	Demand generation	TC-DEM-01 to 05
SCM-F-021	Customer order	TC-ORD-06 to 09
SCM-F-022	Customer order book	TC-ORD-10 to 13
SCM-F-023	FIFO allocation and backorders	TC-ORD-14 to 17
SCM-F-024	Moving average forecast	TC-FC-01 to 05
SCM-F-025	Exponential smoothing, MAE, MAPE	TC-FC-06 to 10
SCM-F-030	Simulation clock and event log	TC-CLK-01 to 05
SCM-F-031	Configuration loading and validation	TC-CFG-01 to 06
SCM-F-032	Transport cost	TC-SHP-01 to 04
SCM-F-033	KPI calculation	TC-KPI-01 to 05
SCM-F-034	CSV export and summary	TC-RPT-01 to 05
SCM-F-035	Simulation engine and CLI	TC-SIM-01 to 06
SCM-NF-001	Performance	TC-NFR-01
SCM-NF-002	Repeatability	TC-NFR-02, TC-SIM-03
SCM-NF-003	Testability and CI time	RTM review; CI job duration
SCM-NF-004	CLI usability and errors	TC-SIM-06
SCM-NF-005	Portability	TC-NFR-03
SCM-SR-001	Configuration validation	TC-CFG-02 to 06
SCM-SR-002	Size and run-length limits	TC-CFG-06
SCM-SR-003	CSV formula injection protection	TC-RPT-03, TC-RPT-04
SCM-SR-004	Protected main branch	Ruleset inspection; rejected direct push
SCM-SR-005	Pinned dependencies, no secrets	Pull request review checklist

14. Test Metrics & Reporting

Metrics collected:

- Test cases executed
- Pass percentage
- Fail percentage
- Defect count

- Requirement coverage (requirements with at least one passing test)
- Line coverage and branch coverage of src/
- Defect resolution status

Reports:

- CI status on every pull request and push to main
- Unit Test Report, System Validation Report, Code Coverage Report and Branch Coverage Report at the end of testing

15. Approvals

Role	Name	Signature / Date
Course Coordinator		
QA Lead	D R Rushmitha	
Product Owner and repository owner	Kallepalli Sai Meghana	
Test Engineer	Spoorthi Poornachandra	
Test Engineer	Siri Krishna C R	
Developer and test support	Shraddha	